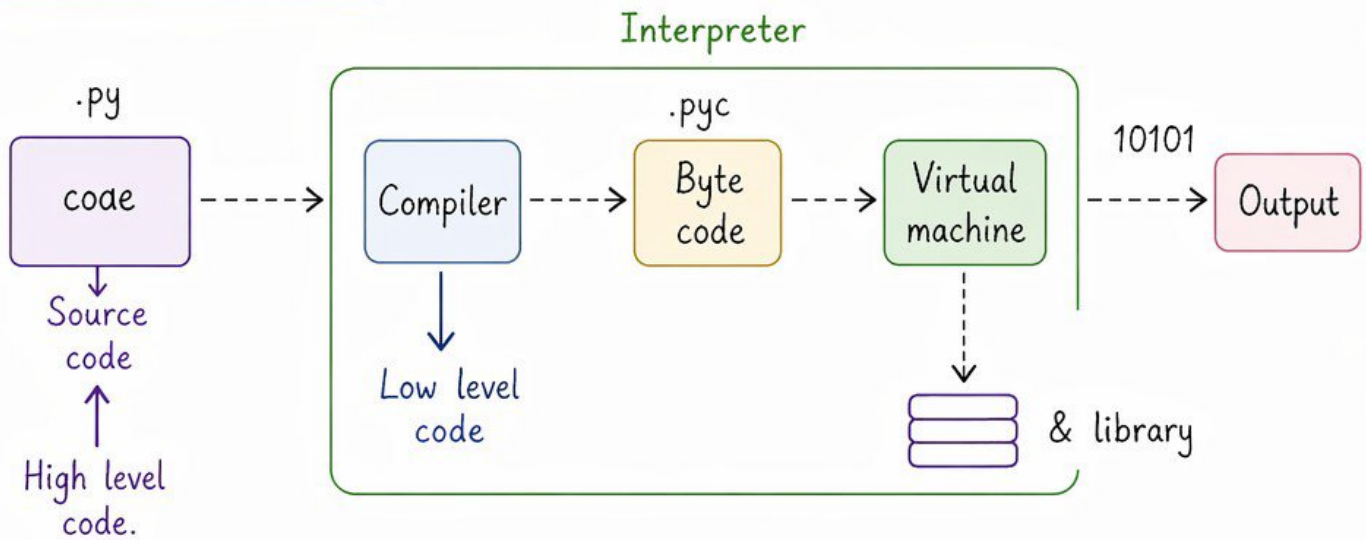


Introduction to Python

1

How Python Works :



Why Python :

- ★ Python is very powerful and simple language compare to other languages
- ★ Python is used literally everywhere, and you can use it in order to build everything.
- ★ Examples – building websites, to work with data, to build games etc.
- ★ Python is high in demand.

Shortcuts :

- We can add shortcuts in visual studio code
 - Shift + Ctrl + P
- Search preferences : keyboard shortcuts

Bonus Upgrades :

- In extensions : we can add Themes
Some popular themes :

One Dark Pro • Ayu • Monokai Pro • Dracula Theme • Material Icon Theme

- Auto Pep8 → A standard format in Python code
↳ Shortcut : Shift + Ctrl + P → search : format document.



What is a comment :-

A comment is a note in your code , python just skips it.

Why we need comments :

- ★ It can make code understandable easily
- ★ It is in professional way

Inline comment $\Rightarrow$ `x = 9    # final exam score.`

Summary :-

- ★ comments are notes in code
- ★ python skips comments
- ★ comments make code :
understandable , Readable , professional.

Print () function :

Print() is a built-in python function that displays messages on the output screen to communicate with users.

Summary :-

- ★ Built-in python function
- ★ Display messages in output for users
- ★ use cases :
communicate , showresult , Debug , Test



Special characters :-

\" → Double Quote
 \' → Single Quote
 \\ → Back slash
 \n → Newline (add newline)
 \t → tab
 \b → Backspace

```
print (" path : C:\user\sai")
```



↳ Error

```
print (" path : C:\\user\\sai")
```

Example :-

```
print (" Hi "python" ")
```

↳ Error



```
print (" Hi \" python " ")
```

(or)

```
print (' Hi "python" ')
```

```
print ("message1\n")
```

```
print ("msg2")
```

o/p:- msg1

msg2

Python challenge :-

→ use print() to recreate exact output, you are allowed to use only one print()

output → you learning path :

- Python Basic
- Data engineer
- AI.

code :-

```
print (" " " your learning path :
```

```
lt- Python Basic
```

```
lt- Data engineer
```

```
lt- AI " " ")
```

""" Triple Quotes :

Allows us to write text in multiple line strings.

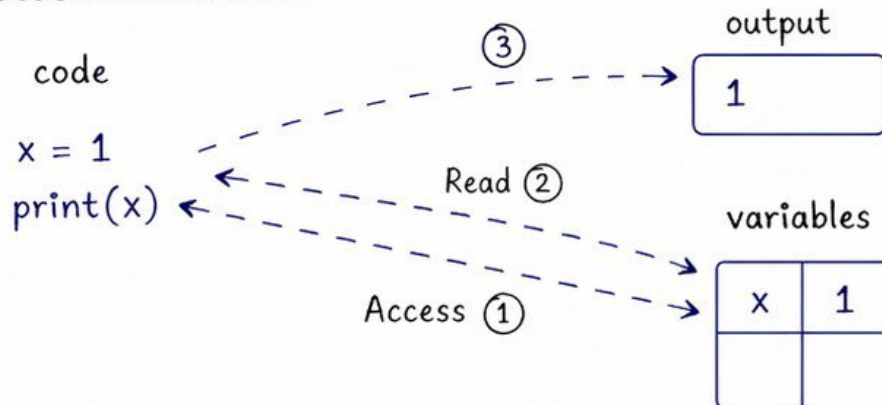


Variables :

What is a variable :

A variable is a name you create to store a value, so you can use it later in your code.

How variable works :



Use of variables :

```
print (" my name is sai")
print (" sai is goodboy")
print (" sai wants engineer")
```

If we want change the name from sai to RAM so It take more time and clumsy to do.

But using variables is so easy.

variable name → `x = "sam"` → value.

```
print (" my name is {x}")
print (" x , is good boy")
print (" x wants engineer")
```

By using this we can change easily.

Variables make updates super easy one change updates everything.

```
name = " sai "
language = " python "
```

→ staitcpart
 → (" name") → variable.

```
print (" my name is", name)
print (" name, " is learning", language)
print ( name, " want to become", python, "engineer")
```



Variables summary :

- ★ Make programs dynamic
- ★ Name to store a value
- ★ Stored in memory
- ★ Reusable anytime
- ★ Updatable anytime

Python challenge :-

- Print following 3 lines
- Add a variable to make it dynamic

o/p → "info @ data with baraa.com"
 "support @ data with baraa.com"
 "www. data with baraa.com."

code :-

```
name = " data with baraa.com"
print ( " info @", name)
print ( " support @", name)
print ( " www.", name)
```

if we write like this

The output have containing space.
 so we & use some methods to
 get exact output.

① remove space we can use:

Method-1 :- string concatenation

```
print ( " info @" + name)
```

Method-2 :- use sep = " "

```
print ( " info @", name, sep = " ")
```

Method-3 :- f-string

```
print (f " info @{name}")
```

code :-

```
name = " data with baraa.com"
```

```
print ( " info @" + name)
```

```
print ( " support @", name, sep = " ")
```

```
print ( f " www.{name}")
```

bestway

↳ Method-3 :- f-string

```
print (f " info @{name}")
```



print() → Display something to user

input() → Get something from user

} Two important
function's.

What is input() ? :-

A built-in python function that
user input

your program to get

```
input (" Enter your name")
```

input() and variable :-

using input() alone reads the user response but immediately
discards it

→ To keep the value, assign it to a variable

```
name = input (" Enter your name")
```

Hard coded values vs dynamic values :-

Hard coded value (static) :-

Fixed piece of data written directly into your code
that never changes at runtime

ex:- country = "India "

Dynamic values :-

data entered by the user that can vary each time
the program Runs

ex:- name = input (" Enter your name")

ex prog :-

```
name = input (" Enter your name")
```

```
country = " Germany "
```

```
print (name, " comes from", country)
```

★ input() makes your prog interactive



Data types :

What is a Data type ?

A datatype is a classification that specifies what type of value a variable can store and what operations can be performed on it

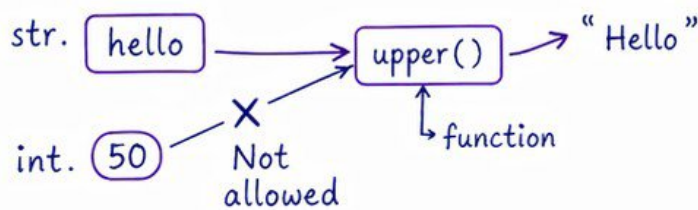
Simple meaning :

It tells the computer :

- what kind of data is (num, text etc)
- How to handle the data

Why datatype ?

- Know how to operate data

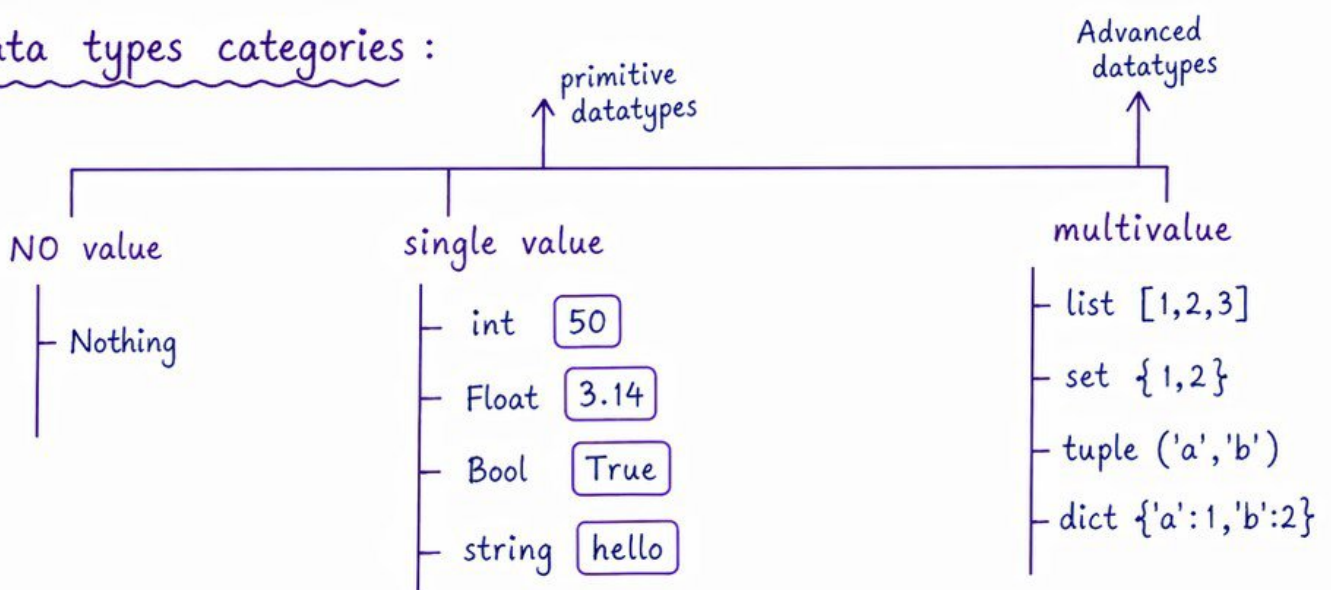


int int
 $2 + 3 = 5$
 Adding numbers

str str
 $"2" + "3" = "23"$
 Joining numbers

- ★ Python automatically detects datatype
- ★ Dynamic : Data types can change Any time.

Data types categories :



data types :-

$a = 10$ # int $\rightarrow$ whole number without decimal

$b = 3.15$ # Float $\rightarrow$ numbers with decimal points

`c = "Hello"`
`d = 'Hi'`
`e = "1234"` } # string → represent text (or) a sequence of characters written inside single (or) double quotes

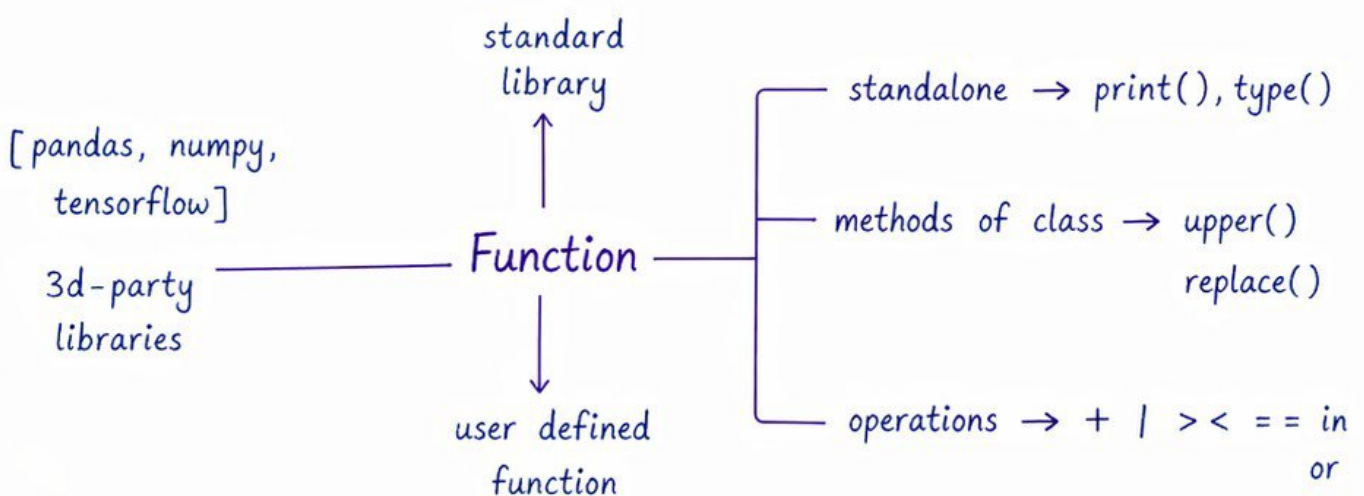
$f = \text{True}$
 $g = \text{False}$ } # bool $\rightarrow$ can be either True (or) False
used to handle logic and decision making

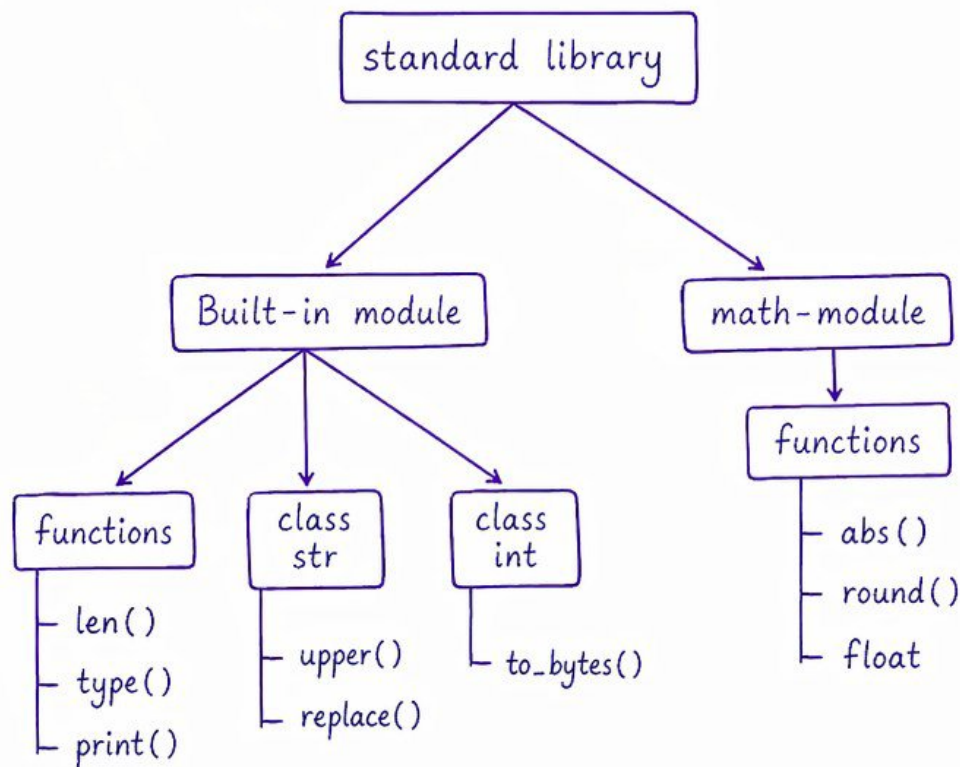
`h = None` → means no value, "nothing" (or) none
It is used to absence of any data

`i = " "` → Blank " " is a string value with no character inside, it is not same as None.

Functions:-

Value $\rightarrow$ Function $\rightarrow$ new value.





Functions :

- * independent block of code

syntax:

function_name (value)

ex: print ("Hello")
 type (50)

Methods:

- * functions belong to objects / classes

syntax:

value.method_name ()

ex: 'hello'.upper ()
 50 . bit_len ()

Summary :

- * Each value has a datatype handle the value
 This helps python decide how to handle the value
- * It automatically detects datatype
- * dynamic : Data types can change anytime
- * 3 categories :
 - NO value : nonetype
 - single value : int, str, bool, float, complex
 - multi value : set, tuple, dict, list
- * values are objects of datatypes class

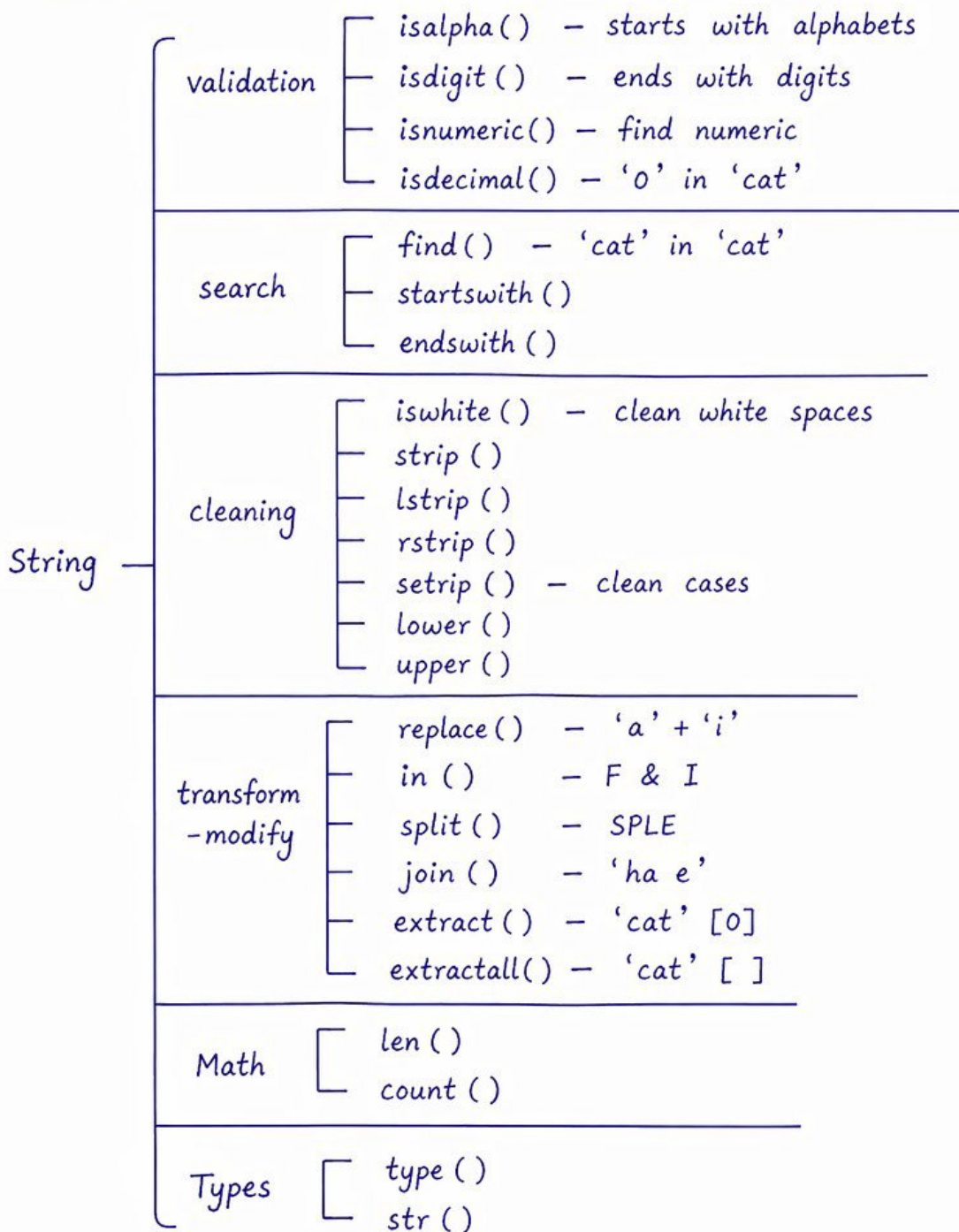


Why is handling string data is important?:-

Text is everywhere

* text data often has poor quality

- File names
- User input
- Web data
- Logs

String function categories:-

Built-in Function :-

11

TYPE (obj) :-

returns the datatype of a value,
so you know what kind of object is

```
Name = "Dinesh"  
print (type(name))
```

o/p:- < class 'str' >

Note:- can't combine a string with an integer using + operator

str (value) :-

converts any value into string value

```
Ex:- age = 24  
print (type(age))  
print ("your age is:" + str(age))
```

o/p:-
< class 'int' >
< class 'str' >
your age is : 24

string function math :-

len (value) :-

returns no. of items in a value
returns the no. of characters in a string

```
password = "1234a"  
print (len(password))
```

o/p:- 5

len count everything even spaces

usecase: validate input length

prevents value that are too short or too long.

count (substring) :-

return how often a word appears in the string

syntax:- text.count(value)

python is case sensitive and means uppercase and lowercase letters are treated as different

EX:- text = ""

python is easy to learn

python is powerful

many people love python ""
count case

print (text.count("python"))

O/P:- 2

use case:-

- * Detect quality issues
- * count How many unwanted characters in my data.

DATA TRANSFORMATION :-

Replace (old, new) :-

swaps part of the text with something new

EX:- phone = "176-1234-56"

print (phone.replace("-", "/"))

O/P:- 176/1234/56

replace() is not just for changing values you can also remove unwanted parts by replacing them with an empty string("")

Chained methods are executed in order from left to right each replace() runs on the result of the one before it

EX:- price = "\$1299.99"

print (price.replace("\$", "").replace(".", ""))

O/P:- 1299.99

Python challenge :-

Convert the messy phone number into a clean number format with only digits

" +49 (176) 123-4567 "

O/P :- ph = "+49 (176) 123-4567"
 print (ph.replace("+", "").replace("(", "").replace(")", "").
 replace("-", "").replace(" ", ""))

491761234567

'string' + 'string' (joins [concatenates] two strings into one)

first_name = " michael"

last_name = "scott"

last_name = first_name + "-" + last_name

print (last_name)

O/P :- michael-scott

usecase :-

Build file paths (build dynamic paths using folder and file variable)

f-string :-

f-strings are modern, super-easy way to format and build strings

"f" stands for "formatted"

lets you easily put variables and expressions directly inside the string value.

sample code :-

```
name = "Sam"
age = 34
is_student = False
```

NOTE :-

can only concatenate only string values (not int)

without f-string :-

```
print (" my name is " + name + "I, am " + str(age) +
      " years old, and student status is " + str(is_student) + "")
```

with f-string :-

```
print (f"my name is {name}, I am {age} years old, and
      student status is {is_student}.")
```

O/P is
same for 2

```
print (f" 2+3 = {2+3}")
```

f-string :-

O/P:- My name is sam, I am 34 years old, and student status is False.

$$2+3 = 5$$

↓
print (f" add this is me {2+3}")

O/P:- add this is me 5

split() :-

① value
 " Adam - 24 - USA "
 ⑥ ② ③
 ["Adam", 24, 'USA'] ← op split('-')
 be a list

split (separator) breaks a string into smaller parts of strings.

O/P:- list of strings

sample code :-

```
stamp = "2026-09-20 14:30"
```

```
print ( stamp.split (" "))
```

O/P:- ['2026-09-20', '14:30']

Now Break a date into year, month, and day parts

```
stamp = "2026-09-20"
```

```
print ( stamp.split ("-"))
```

O/P:- ['2026', '09', '20']

String Repetition :- ('string' * number)

repeats the string multiple times

output: string

usecase:-

use repeated characters to create a clear sections in output

```
ex:- print ("hai" * 3)      # 'hai' repeated 3 times
      print ("#" * 10)     # '#' repeated 10 times
```

O/P:- hahahaha
 #####

Indexing & Slicing :-

h e l l o
↑ ↑ ↑ ↑ ↑

Each character has a position number (index)

+ positive index :-

Starts from left to right

h e l l o
0 1 2 3 4

- negative index :-

Moves from right to left

h e l l o
-5 -4 -3 -2 -1

Indexing :-

↑ "h"
[-5]
[0] ↓ "h"
-5 -4 -3 -2 -1
h e l l o
0 1 2 3 4
↑ "l"
[-3]

Slicing :- [start : end]

↑
Included

↑
Not included.

-5 -4 -3 -2 -1
h e l l o
0 1 2 3 4
↑ start ↑ end

(-5 : -3)

o/p :- "he"

[0 : 3] → o/p :- ['h', 'e', 'l']
"hel"

OPEN-ENDED SLICING :- (if you leave start index empty, python starts from index 0)

[1 :] o/p:- "ello"

[: 1] o/p:- "e"

GENERAL FORM :-

[start : end : step]

EX :- [0 : 4 : 2]

↳ it skips every second character.

o/p:- "hl"

INDEXING :-

Extracts one character by position.

text = "python"

print (text [-6]) o/p:- p

print (text [5]) o/p:- n

SLICING :- (start : end : step)

Extracts a part of the string.

date = "2026-09-20"

print (date [0 : 4]) o/p:- 2026

Data cleaning :-

Removing spaces :-

$\xrightarrow{\text{lstrip()}}$ max $\xrightarrow{\text{rstrip()}}$ o/p:- 'max'
 $\downarrow$
 white space
 (or) we can use strip()

- * we want to remove spaces from only left side use lstrip()
- * we want to remove spaces from only rightside use rstrip()
- * we want to remove spaces from both sides use strip()

Use case :-

- * It removes tabs and multiple spaces at start and (or) end, not in the middle
- * NOTE :- only remove spaces at start and end
- * It removes any characters you want from start and end not just spaces.

Case conversion :-

lower() :- (str method)
makes all letters lowercase

upper() :- (str method)
makes all letters uppercase.

Use case :-

- * clear Data for matching

sample code :-

```

text = " Dinesh SAI "
print ( lower (text))
print ( upper (text))

```

o/p:-

```

Dinesh sai
DINESH SAI

```

Python challenge :-

" 968 - Maria, (D@t@ Engineer)); 274.. "

clean the string !

o/p $\Rightarrow$ name : maria | role : data engineer | age : 27

code :-

```
text = "968 - Maria, (D@t@ Engineer)); 274.. "
```

```
name = text.split('-')[1].split(',')[0].replace("@", "a")
```

```
role = text.split(',')[1].split(')')[0]
```

```
age = text.split(')')[1].split('..')[0]
```

```
result = "name : " + name + " | role : " + role + " | age : " + age
```

```
print(result)
```

o/p :-

name : maria | role : data engineer | age : 27

Searching :-

1. startswith (substring)

checks if string begins with a specific word.

```
phone = "+49-176-12345"
```

o/p:- TRUE

```
print ( phone.startswith("+49"))
```

2. endswith (substring) :

o/p:- FALSE

```
email = "baraa@outlook.com"
```

```
print ( email.endswith("@gmail.com"))
```

3. To check the text is valid (or) not

'substring' IN 'string' :-

checks if a word exist in a string

```
email = "baraa@gmail.com"
```

o/p:- TRUE

```
print ("@" IN email)
```

Find() is the great when combined with other methods to add dynamics

Find(substring) :-

Returns the starting position of a word in the string

Validating :-

- is alpha() :- output: boolean

checks if the string has only letters

```
country = "USA"
```

o/p :- TRUE

```
print (country.isalpha())
```

```
country = "SA1"
```

o/p :- FALSE

```
print (country.isalpha())
```

- is numeric() :- output: boolean

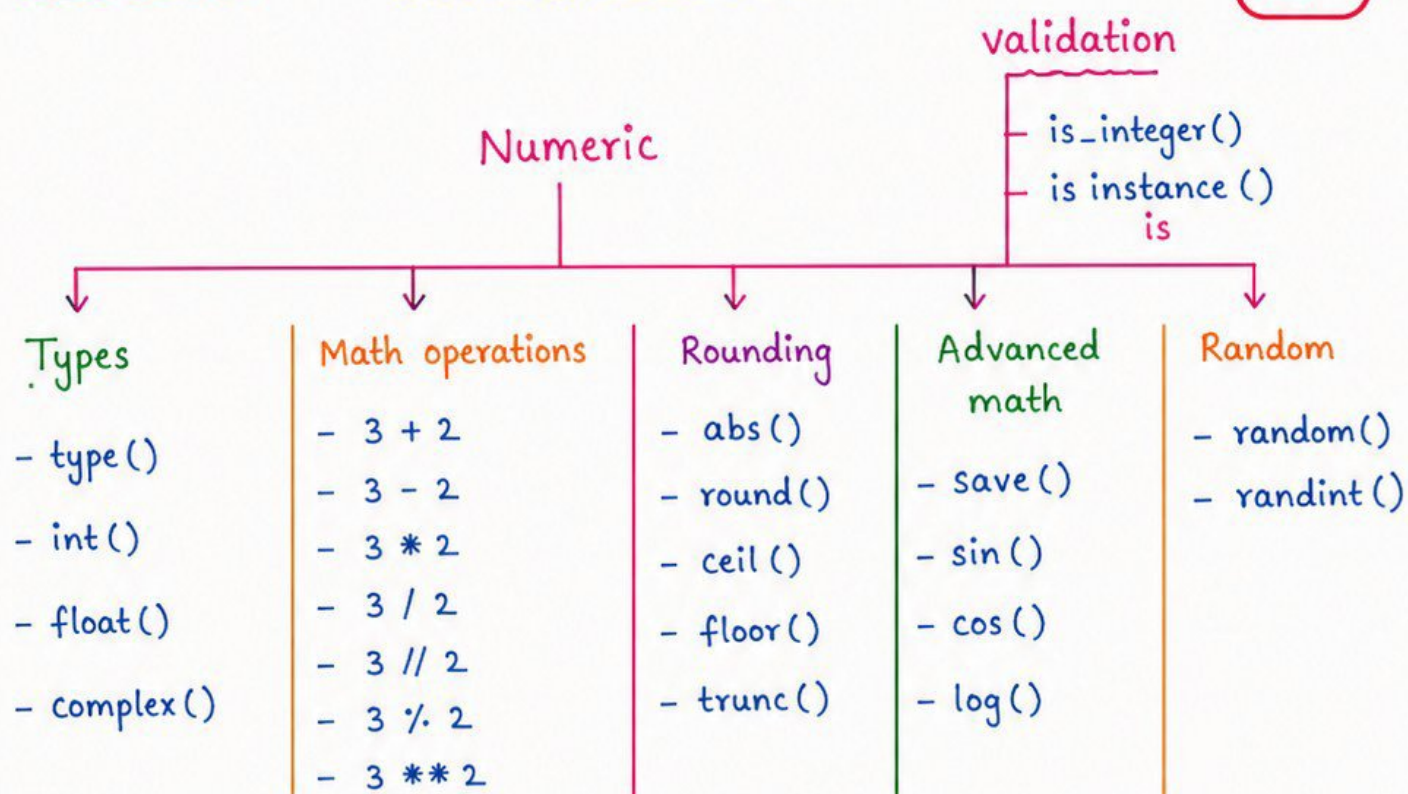
checks if the string has only numbers

```
phone = "01234567"
```

o/p :- TRUE

```
print (phone.isnumeric())
```

NOTE :- Not accept float, special characters, alphabets.

Chapter-3 :-Working with numbers :-

Note: To use numeric we should import math module.

Number functions :-Types :-1. int(value) (built-in function)

converts comparable value into int value

```
x = "24"
print (type(x))
x = int(x)
print (type(x))
```

o/p:- < class 'str' >
< class 'int' >

* We can convert float number into integer

Ex:- x = 3.14
print (int(x))

o/p:- 3

2. float (value) (Built-in function)

converts comparable value into float value.

```
x = 3
print (float(x))
```

o/p:- 3.0

3. complex (real, imag) (Built-in function)

creates a complex number using real and imaginary parts

```
x = 3
```

o/p:- 3+4j

```
y = 4
```

```
print (complex(x,y))
```

o/p:-

5 2 8 3.5 3 1 8

MATH OPERATORS :-

```
print ( 2 + 3 )
```

→ Addition

```
print ( 5 - 3 )
```

→ Subtraction

```
print ( 4 * 2 )
```

→ Multiplication

```
print ( 7 / 2 )
```

→ Floor division (It divides two numbers and rounds down)

```
print ( 7 // 2 )
```

→ Remainder (The left over part after division -

```
print ( 7 % 2 )
```

used to check if a number is even)

```
print ( 2 ** 3 )
```

→ Exponentiation (It raises a number to the power of number)

<operator> = (shorthand assignment)

Apply an operator and reassign the result in one step

```
x = 2
```

```
x += 3 (OR) x = x + 3 (same)
```

```
print (x)
```

o/p:- 5

ROUNDING :-

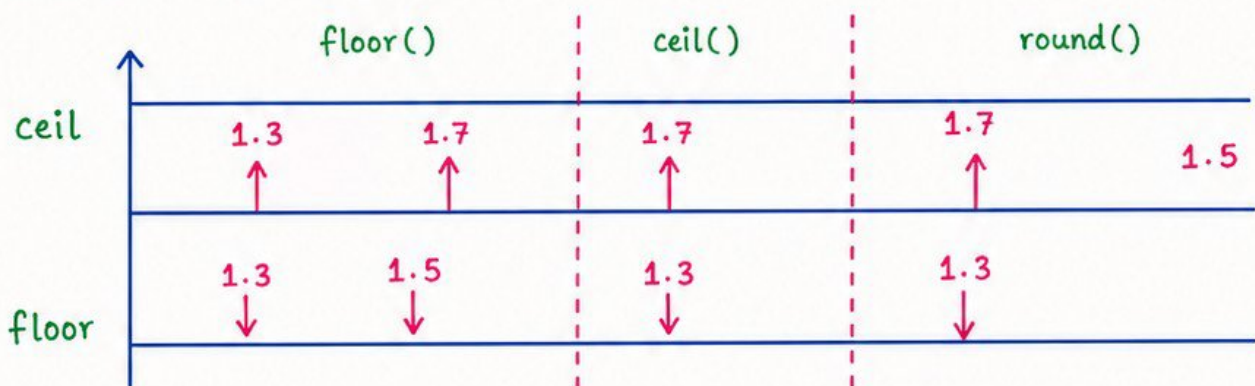
1. abs (value) :- (built-in function)

returns the absolute (non-negative) value of a number

`print ( abs(2-10) )` o/p:- 8

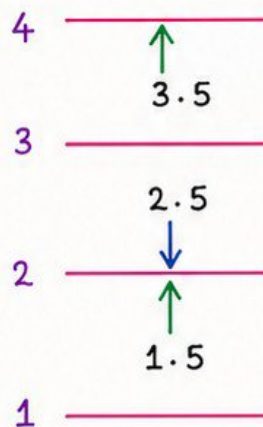
useful for measuring distance (or) size, regardless of direction.

ROUNDING NUMBERS :-



Math. floor(x) :- Rounds a number down the floor

Round half to even :-



In Round it will check whether number is even or not.

NOTE :-

- floor() is not a built-in-function
- floor() belongs to Math module - import it before using

ROUNDING FUNCTIONS (continued)

26

1. math.floor(x) :- (math module)

Rounds a number down to the floor
(greatest integer less than or equal to x)

Ex:- $x = 3.15689$

print (math.floor(x)) o/p:- 3

2. math.ceil(x) :- (math module)

Rounds a number up to the ceiling
(smallest integer greater than or equal to x)

Ex:- $x = 3.15689$

print (math.ceil(x)) o/p:- 4

3. round(x) :- (built-in function)

Rounds a number to the nearest whole number
up or down depending on what's closer.

Ex:-

$x = 3.15689$

print (round(x))

o/p:- 3

How it works ?

- If decimal part $< 0.5 \rightarrow$ Round down
- If decimal part $> 0.5 \rightarrow$ Round up
- If decimal part $= 0.5 \rightarrow$
Round to the nearest **EVEN** number
(Banker's Rounding)

Banker's Rounding (Round half to even) :-

If the decimal part is exactly 0.5, it rounds to the nearest even number.

Examples :-	$1.5 \rightarrow 2$ (2 is even)	$2.5 \rightarrow 2$ (2 is even)	$3.5 \rightarrow 4$ (4 is even)	$4.5 \rightarrow 4$ (4 is even)	$5.5 \rightarrow 6$ (6 is even)	$6.5 \rightarrow 6$ (6 is even)
-------------	------------------------------------	------------------------------------	------------------------------------	------------------------------------	------------------------------------	------------------------------------

More Examples :-

round (2.3) = 2

round (3.5) = 4 (even)

round (2.7) = 3

round (4.5) = 4 (even)

round (-1.4) = -1

round (7.5) = 8 (even)

round (-1.6) = -2

round (8.5) = 8 (even)

Remember :-

When decimal part is 0.5
it rounds to the
nearest **EVEN** number.

4. round(number, ndigits) :- (built-in function)

Rounds number to the specified number of decimal places.

Ex:- price = 35.5468979

print (round (price, 2)) o/p:- 35.55

ROUNDING FUNCTIONS (continued)

27

4. math.trunc(x) :- (math module)

Cuts off the decimal parts and gives (keeps) the whole number (no rounding)

Ex:- price = 35.548798
 print (math.trunc(price)) o/p:- 35
 print (int(price)) 35

int() vs trunc()

int() → Converts to integer,
may cause issues if number
is very large (overflow).

trunc() → Always returns the
integer part without any
rounding or conversion issues.

- * If you're not using math already,
Just use **int()** it's simple and built-in.
- * If you've already imported math,
use **trunc()** it makes your intent clearer.
- * Use **int()** for general purpose conversion.
Use **trunc()** when clarity is important.

Use:-

round() → Handy in data analysis to clean up numbers
for reports or save spaces

ceil() → Perfect for data engineering - like splitting data
into pages or batches.

floor() → Useful in pagination, division of items, etc.

trunc() → Useful when you just want to remove decimal part
without rounding.

RANDOM :-

1. random() :- (random function)

returns a random float between 0.0 and 1.0

import random

Ex:- print(random.random()) o/p:- 0.798....

2. randint(start, end) :- (randint function)

gets a random whole number from start to end
(both included)

import random

Ex:- print(random.randint(1,6)) o/p:- 5

→ Use it to generate test data (dummy)
for like age, ID, or prices.

VALIDATION :-

1. is_integer() :- (float class method) (is a whole number)

checks if a float has no decimal parts

Ex:- x = 7.0

print(x.is_integer()) o/p:- TRUE

y = 7.1

print(y.is_integer()) o/p:- FALSE

Use:-

- Checks if numbers are truly whole
- Floats with .0 might just be from file exports

INSTANCE (value, type) :- (built-in function)

→ Checks if a value belongs to a certain datatype.

Ex:- `x = 70`

o/p:- **TRUE**

`print(instance(x, int))`

Syntax :-

`instance(value, type)`

- value → the value to check
- type → the datatype (class)

Python Challenge :-

Generate a random integer b/w 1 and 100, and check if result is an even number.

`import random`

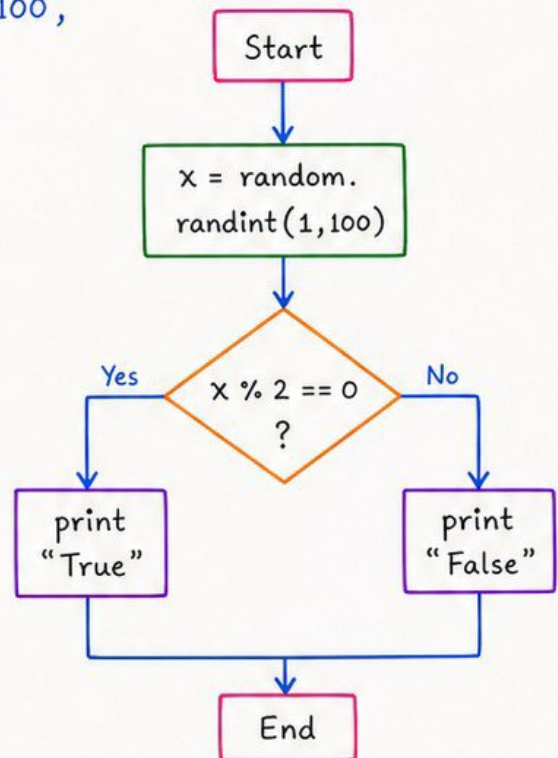
`x = random.randint(1, 100)`

`print(x)`

`if (x % 2 == 0):`

`print("True")`

o/p:- 46
TRUE



Use :-

- `random()` → Useful for simulations, games, lottery, etc.
- `randint()` → Useful for generating test data, IDs, ranges, etc.
- `instance()` → Useful for type checking and validating data.

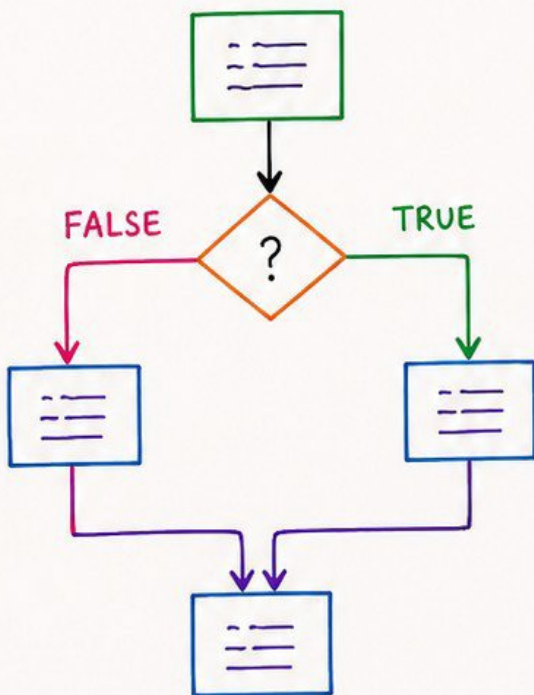
CONTROL FLOW :-

It is the logic you write to control how your code runs.

" should I run this part of the code "

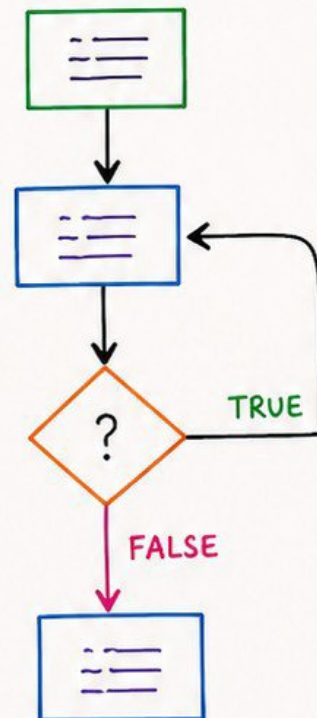
" should I skip it "

" should I repeat it? "

CONDITIONAL STATEMENTS

To run only one either **TRUE** or **FALSE**. Based on decision.

make decision

LOOPS

To repeat until certain condition is **TRUE**.

Repeat task.

SPECIAL INSTRUCTIONS
IN PYTHON TO CHANGE
THE FLOW

CONTROL FLOW

CONTROL FLOW STATEMENT

• CONDITIONAL STATEMENTS

if else elif

LOOPS TYPES → for while

CONTROL

break pass continue

BOOLEAN EXPRESSION

VALUES

TRUE FALSE

FUNCTIONS

bool() any()

all() isinstance()

MEMBERSHIP

in not in

LOGICAL OPERATORS

and or not

COMPARISON OPERATORS

== != <

> >= <=

IDENTITY

is is not

BUILD QUESTIONS (CONDITIONS)

Python can evaluate as "yes" or "no" (True or False)

BOOLEAN :-

1. bool (value) :- (built-in function)

TRUE → if the value is non-empty or non-zero

FALSE → if the value is empty or zero

EX:-

print (bool (123))

print (bool ("123"))

print (bool ())

print (bool (0))

O/P:-

TRUE

TRUE

FALSE

FALSE

NOTE :-

- **None** means no value
- **Empty** means the value exist, but empty string

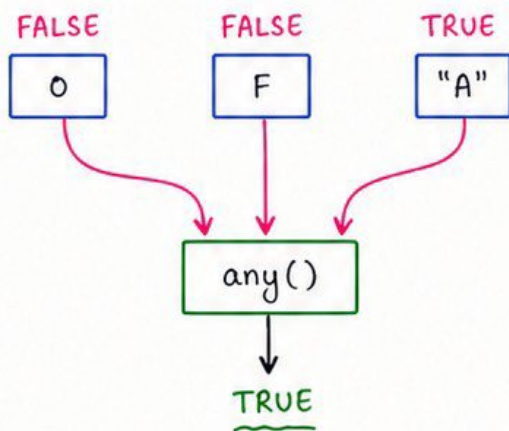
NOTE :-

None means no value.

Empty means the value exist, but empty string.

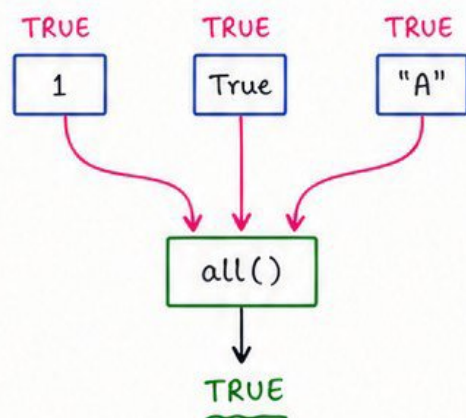
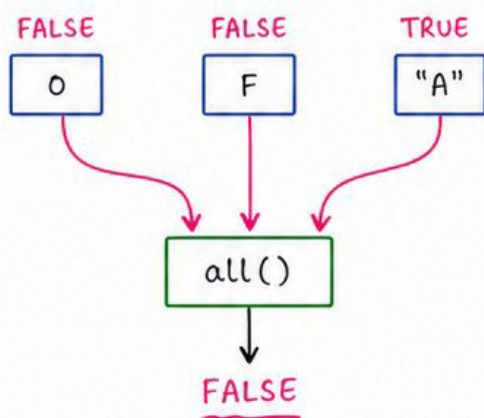
any() :- (built-in function)

Returns True if at least one value is TRUE.



all() :- (built-in function)

Returns True if all values are TRUE.



EX :-

Email = ""

Phone = "012-487"

user = "Dinesh123"

print (any (Email, Phone, user))

print (all (Email, Phone, user))

O/P :-

TRUE

FALSE

NOTE :-

- **None** means no value.
- **Empty** means the value exist, but empty string.

COMPARISON OPERATORS :-

It compare two values and return **True** (or) **False**

Based on the result.

Value operator value ? $\begin{cases} \text{True} \\ \text{False} \end{cases}$

value 3 > 2 $\Rightarrow$ **True**

variable x < 2 $\Rightarrow$ **False** (If x = 5)

expression 2 - 1 != 2 $\Rightarrow$ **False**

function $\underbrace{\text{len}("Hi")}_2 == 3$ $\Rightarrow$ **False**

COMMON COMPARISON OPERATORS

==	Equal to
!=	Not equal to
>	Greater than
>=	Greater than or equal to
<	Less than
<=	Less than or equal to

NOTE :-

- Strings can be compared too!
- You can compare strings too alphabetically, not just numbers.

print ("a" == "b") $\rightarrow$ **False**

So "a" and "A" are treated as different values.

Python is case sensitive

= $\rightarrow$ assigns

== $\rightarrow$ compares

CHAINED COMPARISON :-

Check multiple conditions in one line just like Math.

print (1 < 4 < 6) O/P:- **True**

- $\rightarrow$ It evaluates it from left to right, checking each condition one by one.
- $\rightarrow$ It is **True** only if all conditions are **True**.

LOGICAL OPERATORS:-

Logical operators (and | or) used to combine multiple boolean expressions.

AND :-

- Both must be True.

EX:- $3 < 5$ and $5 == 5 \Rightarrow \text{True}$
 True True

$3 > 5$ and $5 == 5 \Rightarrow \text{False}$
 False True

NOTE :-

AND operator has higher priority than OR.

OR :-

- Atleast one is True.

$3 < 5$ or $5 == 5 \Rightarrow \text{True}$
 True True

$3 > 5$ or $5 == 5 \Rightarrow \text{True}$
 False True

$3 < 5$ or $5 == 4 \Rightarrow \text{True}$
 True False

$3 > 5$ or $5 == 4 \Rightarrow \text{False}$
 False False

NOT OPERATOR :- (It reverse the truth)

- it turns True into False and False into True.

not True $\Rightarrow$ False

not False $\Rightarrow$ True

TASK :-

Allow access only if the user login or they are guest but they must not banned.

is_logged_in = True

is_guest = False

is_banned = True

```
print ( ( is_logged_in or is_guest ) and not is_banned )
```

True	or	True	and not	True
True				

o/p :- False

MEMBERSHIP OPERATOR (in) :-

checks if a value inside another value
like a string, list, tuple or other sequence.

Ex :- print ("f" in False) → True

print (3 in [1,2,3]) → True

TASK :-

Validate that the domain is not on the banned list.

domain = "gmail.com"

banned_domains = ["spam.com", "bot.net", "face.org"]

print (domain not in banned_domains) o/p → False

IDENTITY OPERATOR :-

checks if two variables refer to the same object in memory.

```
X = ['a', 'b', 'c']
```

```
Y = ['a', 'b', 'c']
```

```
print ( x == y )    o/p → True    (compare values)
```

```
print ( x is y )    o/p → False   (compare ids)
```

TASK:-

validate the email address it must be filled in and not empty.

Email =

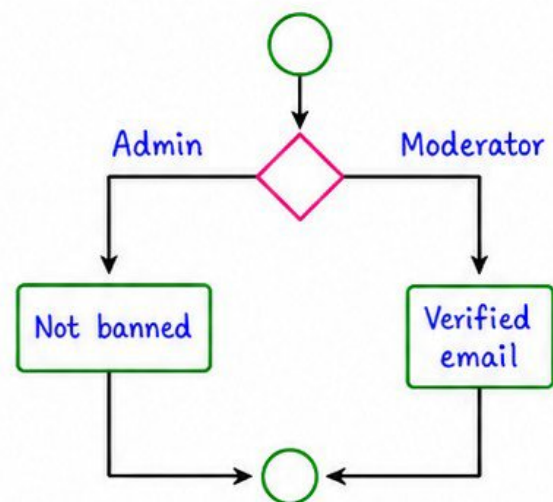
NOTE:-

- None → means NO value at all, it is unknown
- " " → means an empty but it is known, it is string

Use (is) instead of (==) when checking for none.

PYTHON CHALLENGES :-

1. Check if a username is not empty and the age is greater than or equal to 18.
2. Check if the password is atleast 8 characters long and does not contains spaces.
3. Check if a user's email is not empty, contains '@' and ends with '.com'
4. Check if a username is a string, is not None, and is longer than 5 characters
5. Check if the user is either an admin or a moderator and either they are not banned or they have to verified their email.



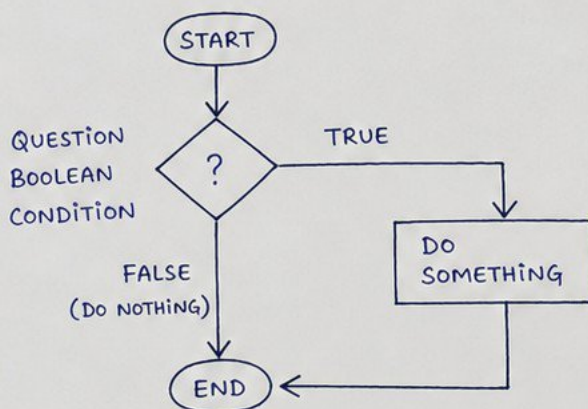
It is a checkpoint in the code that checks a condition

- condition is TRUE ? Runs the code
- condition is FALSE ? Skips it

IF STATEMENT :-

Defines the first condition

(" IF this is true, do this
- otherwise, do nothing")

**IF RULES :-**

- * only one 'if'
- * Start with 'if'
- * required
- * can standalone.

INDENTATION :-

Adding spaces between at the beginning of a line to show that the line belongs to a code block.

- It is part of an syntax.

if True:

 print("Hello") ← inside block
print("Done") ← outside block

NOTE :-

Without indentation

→ **Error** (SyntaxError)

ELSE STATEMENT :-

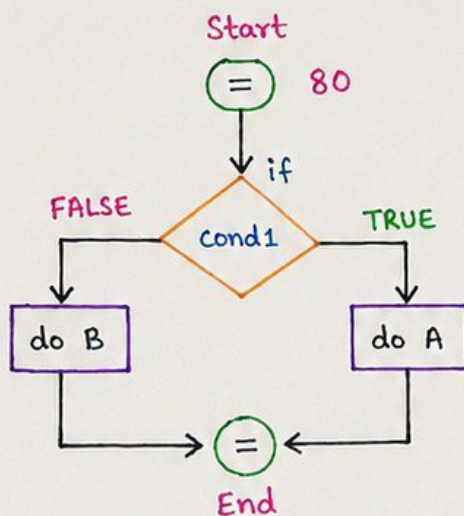
Runs only if all previous conditions are FALSE

if cond1:

do A

else :

do B

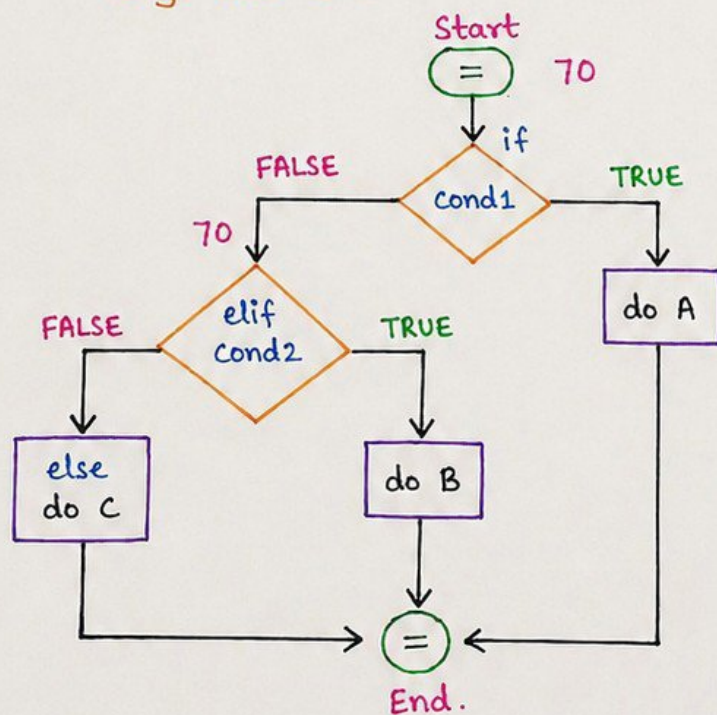
RULES :-

- * Comes at end
- * No conditions
- * Optional
- * Cannot standalone
- * Only one else.

MULTIPLE CONDITIONS (elif) :

Asks a follow-up question only if previous conditions were FALSE.

"if the first wasn't true
try this one"



if cond1:

do A

elif cond2:

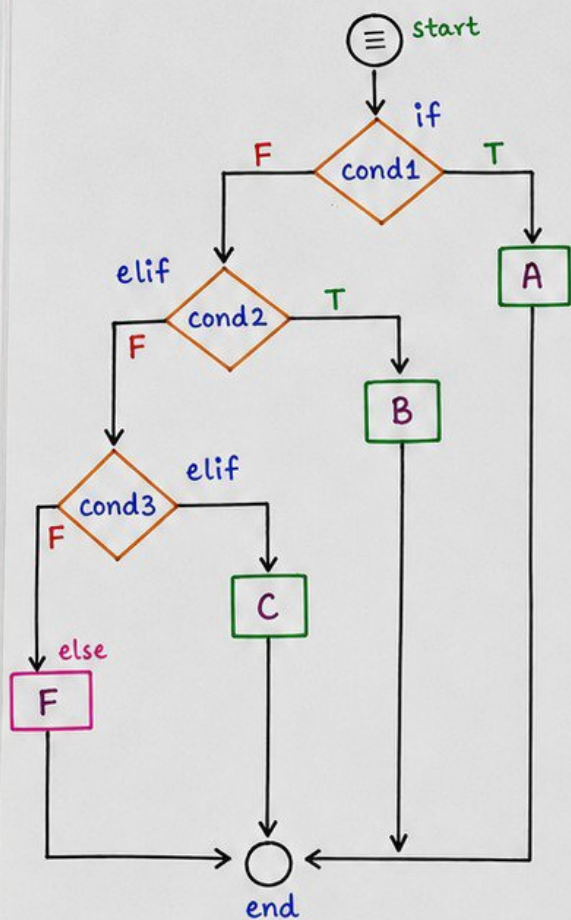
do B

else:

do C

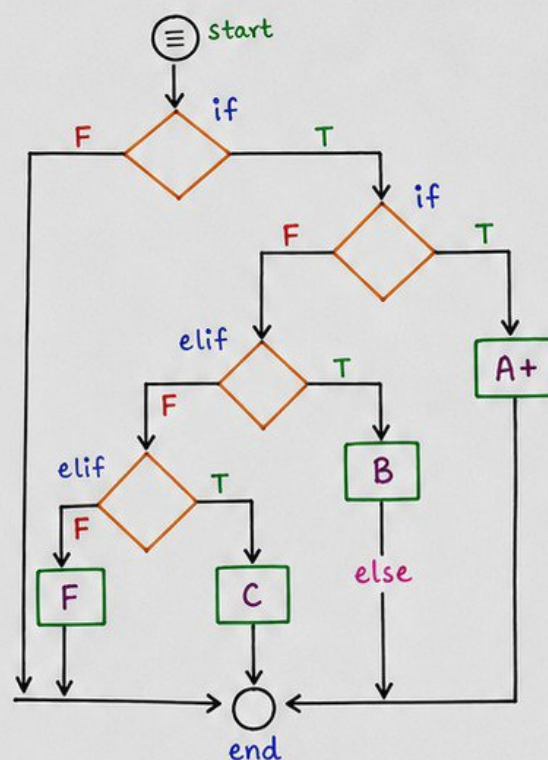
Rules :-

- * Comes after if
- * multiple elif
- * needs condition
- * optional
- * Cannot standalone.

BRANCHING (elif elif ...):-NESTED IF :-

If statement inside another if

"if the first is true, then check the second".

NOTE :-

- * Python evaluates Boolean conditions directly.
- * Avoid explicit comparisons `== True` or `== False`.

EXAMPLE :-

```

if x > 0:
    print("Positive")
elif x == 0:
    print("Zero")
else:
    print("Negative")
  
```

KEY POINTS :-

- * Checks multiple conditions in order.
- * Runs the first TRUE block.
- * Else runs only if all are FALSE.
- * Makes decision making easy & clean.

Sample code :-

```
n = int(input("Enter the value of n :"))
```

```
Submitted_project = TRUE
```

```
if n >= 90:
```

```
    if Submitted_project:
```

```
        print('A+')
```

```
    else:
```

```
        print('A')
```

```
elif n >= 80:
```

```
    print("B")
```

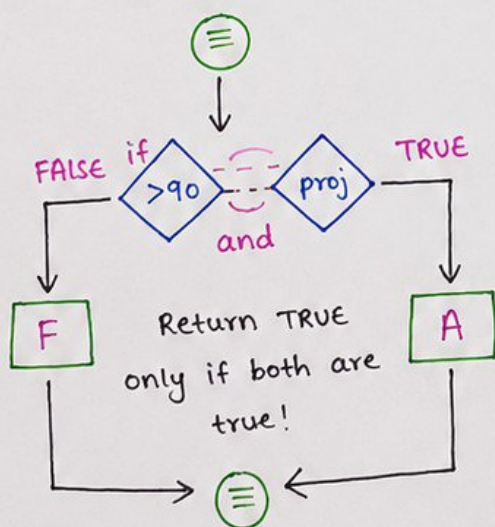
```
elif n >= 70:
```

```
    print("C")
```

```
else:
```

```
    print("F")
```

connecting condition :-



After using connecting statement:-

```
Score = 90
```

```
Submitted_proj = FALSE
```

```
if score >= 90 and Submitted_proj:
```

```
    print("A+")
```

```
elif score >= 90:
```

```
    print("A")
```

```
elif score >= 80:
```

```
    print("B")
```

```
elif score >= 70:
```

```
    print("C")
```

```
else:
```

```
    print("F")
```

Independent IFS :-

Each if is checked separately

"All conditions are tested - even if one is already TRUE."

EX :-

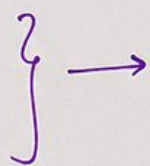
① if :
else :

② if :
else :

Inline if :-

```
if cond1 :
    do A
```

```
else :
    do B
```



inline if

do A if cond1 else do B

Quick & short

Ex:-
X = "A" if score >= 90 else "F"

score = 100

grade = "A" if score >= 90 else "F"

print (grade)

NOTE :- Simple logic = inline-if.
Complex logic = classical-if.

MATCH-CASE :-

Evaluate a value against multiple values,
Runs the code of the first match.

TASK :-

convert the full country names into 2-letter abbreviations.

Ex:-

country = india

if country == "united states"
 print ("us")

else :
 print ("unknown")

Normal way :-

```
Country = "INDIA"
```

```
if country == "united states":
```

```
    print ("us")
```

```
elif country == "India":
```

```
    print ("In")
```

```
elif country == "egypt":
```

```
    print ("Eg")
```

```
else:
```

```
    print ("unknown")
```

- * for flexible logic and multiple conditions.

using match-case :

```
match country:
```

```
    case "united states":
```

```
        print ("us")
```

```
    case "India":
```

```
        print ("In")
```

```
    case "egypt":
```

```
        print ("Eg")
```

```
default:
```

```
    print ("unknown country")
```

use case :-

- * easy & write
- * can be used only for matching values

Summary :-

- if :- starts the 1st condition
- elif :- Adds a follow-up condition if the previous is false
- else :- None of the conditions are met

stand alone if :- Just checking

if-else-statement :- This or that

if-elif-else :- Branching → Choose one from many.

nested IF :- Layered Tree → step-by-step decision

independent IF :- "Checklist mode"

Python challenge:-

Validate the quality and correctness of email values

- Must not be empty
- Must contain ',' and '@',
- Must contain exactly one '@' symbol
- Must end with '.com', '.org' or '.net'
- Must not be longer than 254 char
- Must start and end with a letter or digit.

OUTPUT:-

`isalnum()`:- checks if the string contains only letters and digits.

NOTE:- you want to stop at the first condition that returns true?

if-elif-else.

you want all conditions to be evaluated

Independent if statements.

code :-

```
Email = "baeraa@gmail.com"
```

```
# clean the string
```

```
Email = Email.strip()
```

```
# Email must not be empty
```

```
if Email == "":
```

```
    print("Email cannot be empty")
```

```
# Email must contain a '.' and '@'
```

```
elif not ('.' in Email and '@' in Email):
```

```
    print("Email must contain . and @")
```

```
# Email must contain exactly one @
```

```
elif Email.count('@') != 1:
```

```
    print("Email must contain exactly one @.")
```

```
# Email must end with '.com', '.org', or '.net'
```

```
elif not Email.endswith('.com', '.org', '.net'):
```

```
    print("Email must end with '.com', '.org', or '.net'")
```

Loop :-

Control the flow of code (or)

Repeat a block of code over and over until a condition is met

Types of Loop :-

- i) For Loop.
- ii) While Loop.

For Loop :-

For Loop go through a group of items

Something for each item

for *i* in (range):
 print(i)

loop: variable sequence.

python Iterator :-

An object that lets you go through items one by one in a sequence.

Range (start, stop)

Inside sequence.

not inside sequence.

we can use step also.

range (start, stop, step) → range (0, 10, 2)

o/p :-

0 2 4 6 8
+2 +2 +2 +2

In sequence we can use

- File
- Tuple
- List
- String

Range.
dict.

For Loop in Real world data :-

Scores = [80, 50, 60, 75]

total = 0

for score in scores:

 total += score

 print ("current total:", total)

print ("final total:", total)

NOTE :-

We should use loop variable, not the sequence.

We use for loops to transform data like cleaning data before processing:

47

```
Files = ['REPORT.csv', 'data.csv', 'final.TXT']
```

Here we have inconsistent casing & unnecessary spaces

If a person is Data Engineer & Analyst!!

He should clean the data

for file in Files:

```
file = file.strip().lower().replace('txt', 'csv')
```

```
print(f'processing {file}')
```

O/P:-

```
processing report.csv
" data.csv
" final.csv.
```

Advanced loop:-

When to stop, when to skip, when to break.

Break statement:-

It stops the loop immediately.

It jumps out and ends the loop right away.

ex:-

```
for i in (1,2,3):
    if i == 2:
        break
print(i)
```

O/P:-

```
1
```

Continue Statement:-

It skips one loop cycle without stopping the loop skips this one and go next.

EX:-

```
names = ['john', 'maria', '-', 'kumar']
for name in names:
    if name == '':
        print('Empty value detected!')
        continue
    print(f'Name - {name}')
```

pass statement:-

It is a place holder where nothing happens.

For now.. just keep going do nothing...

It is just place holder to be changed later.

```
names = ['A', 'B', '', 'D']
```

```
for name in names:
```

```
    if name == '':
```

```
        pass # Handle empty value
```

```
        name = name.replace('', 'unknown')
```

```
    print(f'Name = {name}')
```

O/P:-

NAME = A

" = B

" = unknown

" = D.

NOTE:-

USE "ELSE" with loops only when there is a break.

While loop:-

Repeats a block of code - over and over as long as condition is **TRUE**!

49

```
i = 1           ← initialization
while i < 4 :   ← condition
    print(i)    ← update
    i = i + 1
```

If we don't use these 3 we will stuck in infinity loop.

TASK:-

Write a prog that keeps asking "Do you agree?" until the user types "yes".

Sol:-

```
answer = ""
```

```
while answer != "yes" (Do you agree?(yes/no):).
```

```
    answer = input("Do you agree?")
```

```
    print("Thank you")
```

} exist normally
after "
yes.

While TRUE:- (most powerful and risky).

While TRUE:

```
x = input("type")
```

```
if x == 'stop':
```

```
    break.
```

- Must have extra **if+break**
Risk in infinite loop.

Advanced Data Types :-

IMP

50

CHAPTER - 7 :- DATA STRUCTURES

DATA STRUCTURES :-

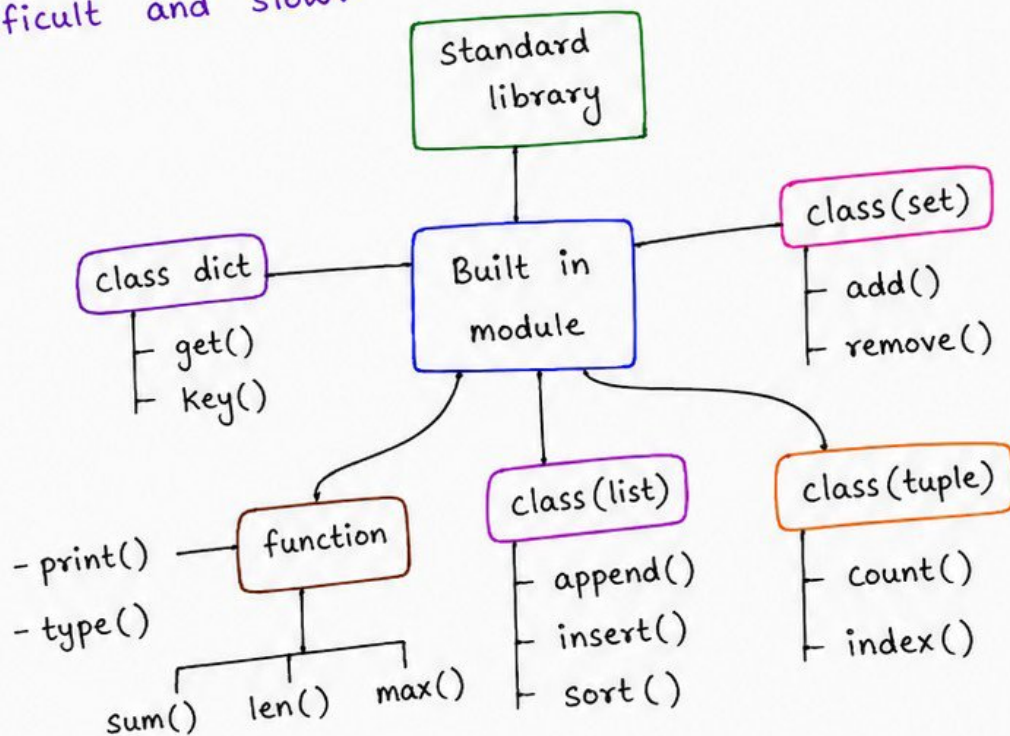
It is a way of organizing and storing data so it can be used efficiently.

- 1) List [] → common
- 2) tuple () → no change
- 3) set {} → unique
- 4) dict {} → Key value

WHY DATA STRUCTURES :-

We need data structures in python to store, organize and manage data efficiently.

Without data structures, handling large amounts of data become difficult and slow.



Functions vs Methods :-

51

Function

costs = (10, 20, 30)

len(costs)

Input → len() → output
[10, 20, 30] 3

Method

costs = (10, 20, 15)

costs.remove(10)

Input → remove → Output
(10, 20, 15) None.

flexible

DATA STRUCTURE LISTS[] :-

- * A List is ordered collection of items. It is very changeable
- * Allow duplicates
- * Widely used in python.

Basic Operations :-

- How to create ?
- How to access & read ?
- How to unpack ?
- How to explore & analyze ?
- How to change ?
- How to organized ?
- How to copy ?
- How to combine ?

Advanced :-

- How to iterate ?
- How to transform ?
- How to filter ?
- List Comprehension ?

create Lists :-

- create an empty List ?

```
empty = []
```

```
print (empty)
```

```
print (type (list))
```

o/p:- []

<class 'list'>

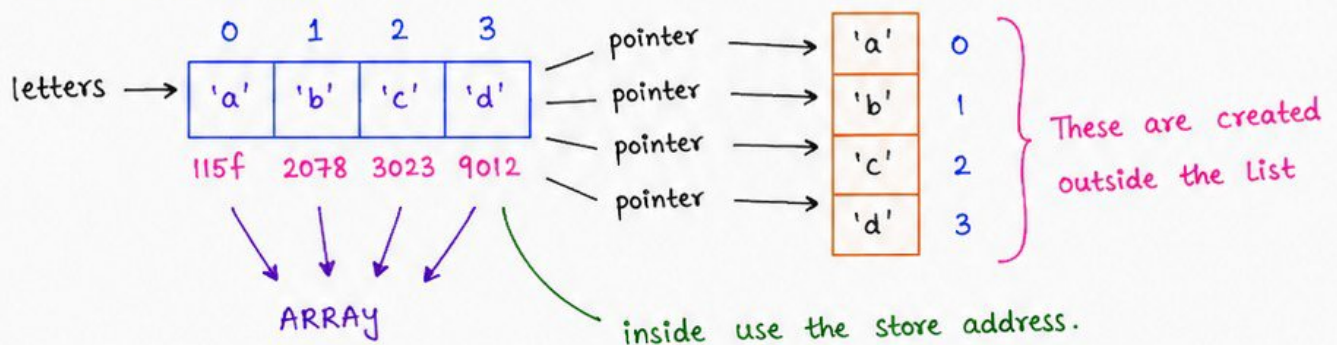
- create a List by adding items manually?

```
letters = ['a', 'b', 'c', 'd']
```

o/p:- ['a', 'b', 'c', 'd']

```
print (letters)
```

- Behind The scenes:-



* We can mix datatypes in the same List.

```
mixed = [1, 'a', TRUE, FALSE]
```

By using Built-in-Function:-

List (Value):- built-in-function

converts an iterable (sequence) into a list.

```
letters = list ('python')
```

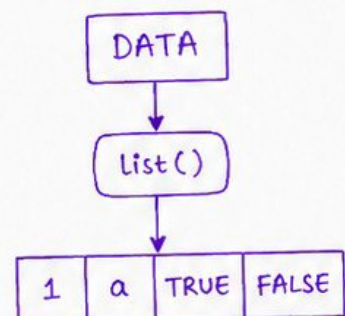
```
print (letters)
```

o/p:- ['p', 'y', 't', 'h', 'o', 'n']

```
numbers = list (range(5))
```

```
print (numbers)
```

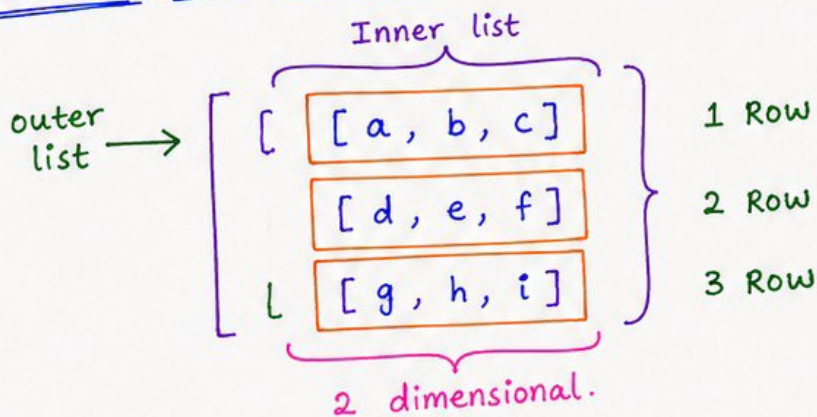
o/p:- [0, 1, 2, 3, 4]



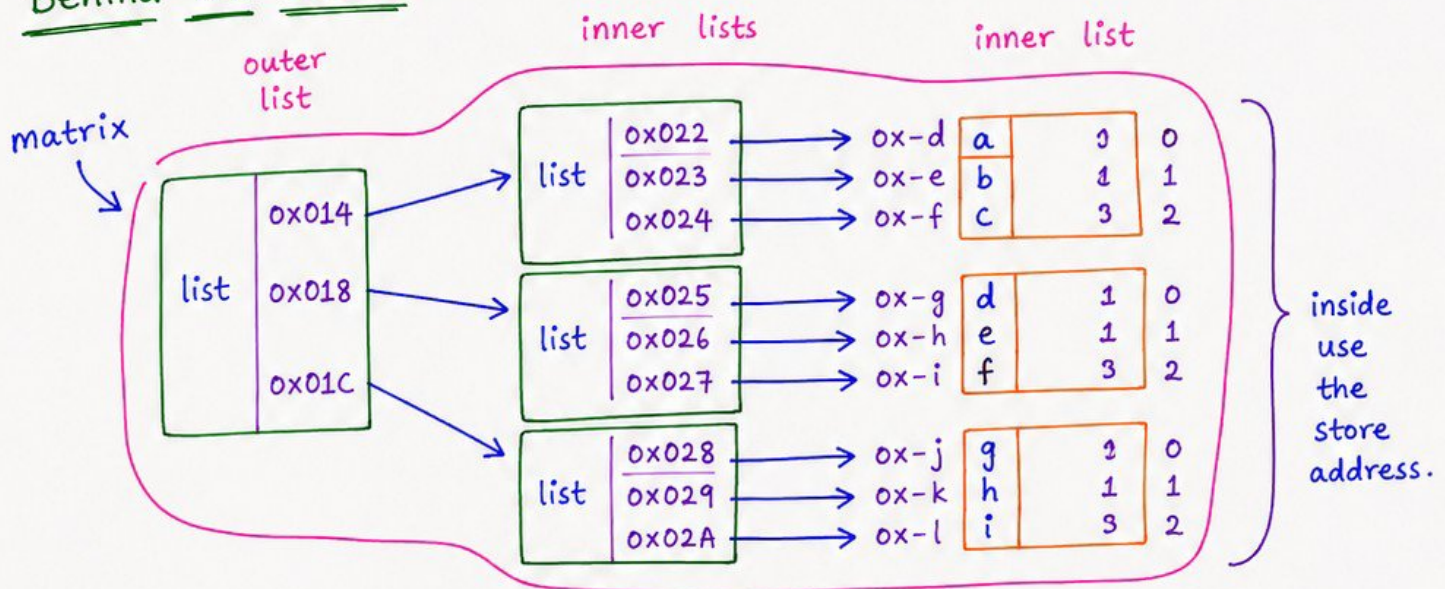
Nested Lists :- (MATRIX)

53

Nested List Matrix :-



Behind the scenes:-



* We can mix datatypes in the same list.

`mixed = [ 1, 'a', TRUE, FALSE ]`

Recap:-

Empty List

`x = []`
`x = list()`

Manual creation:-

`x = ('a', 'b', 'c')`

Nested List :-

`x = ([a, 'b', 'c'], ['c', 'd'])`

Read & Access :- (LISTS[])

54

Get First Item from the list

```
lst = ['a', 'b', 'c', 'd']
```

```
print (lst[0]) # Get first item from the list
```

```
print (lst[-1]) # Get last item from the list
```

O/p:- a
 d

NOTE:- use -ve indexes to access items from the end of list.

Get c from the list

```
print (lst[-2])
```

Access & Read Nested List (MATRIX) :- (Indexing)

	0	1	2
Row 0	a	b	c
Row 1	d	e	f
Row 2	g	h	i

`print (MATRIX)` → $\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$ — whole matrix

`print (MATRIX[1])` → d e f — complex row

`print (MATRIX[1][1])` → e — one single value.

Slicing :- To print 2 elements in matrix

First (Index) → default.

```
lst = ['a', 'b', 'c', 'd']
```

```
print (lst[:2]) # Get first two items from lst
```

```
print (lst[2:]) # Get last two items from lst
```

```
print (lst[:]) # To print whole list
```

O/p:-

`['a', 'b']`

`['c', 'd']`

`['a', 'b', 'c', 'd']`

```
Mt = [['a', 'b', 'c'], (1, 2, 3), (True, False, 'NOT')]
```

```
print (matrix [0:2]) # Get first two rows (lists)
```

```
print (matrix [1: ]) # Get the last 2 rows (lists) from Mt
```

To slice inside one specific row

```
print (matrix [2][ : 2]) o/p:- [True, False]
```

SUMMARY:-

How to Access & Read ?

Store :

-2	-1
'a'	'b'
0	1

- Indexing : get 1 single item $X[1]$
- Slicing : get multiple items $X[1:3]$

UNPACKING LISTS:-

- We can do in python with our list. As you create your list, you are packing your items inside a container and we can use unpacking in order to take the items out of the list and put it in different variables.

Ex:- person = ['Maria', 29, 'Data Engineer', 'Spain']

without unpacking:

name = person [0]

Age = person [1]

Role = person [2]

Country = person [3]

Without unpacking using only indexes make code long and hard to extend.

56

By using unpacking:

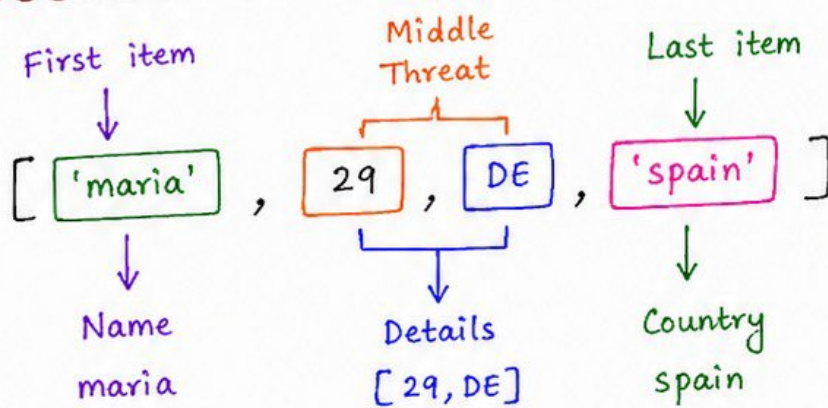
Use of variables, separated by commas
name, age, role, country = person.

print(name)
o/p: Maria.

NOTE:-

- * Variable order must match the list values order simple to.
- * Print unpacking is clean, easy and makes code extend.

'Rest collector' Asterisk (*) :-



person = ['maria', 29, 'DE', 'spain']

Ex:- person = ['maria', 29, 'DE', 'spain']

name, *details, country = person

print(name) o/p: Maria

print(details) o/p: [29, DE]

NOTE:-

It is used when we want 1st and last elements we use rest.

Only one asterisk (*) is allowed in unpacking.

Unpacking Rules:- (*) → stores the rest in new list.

57

1. No. of variables must match the values exactly - not less, not more.
2. Asterisk (*) collect leftovers, and it's fine if there are none.

numbers = []

o/p:- []

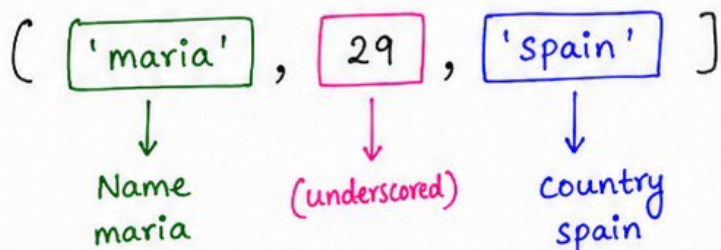
first, *rest = numbers

(list, tuple, strings, etc)

You can unpack any sequence.

Skipping Items underscores ("_")

It is used to skip item



person = ['maria', 29, 'DE', 'spain']

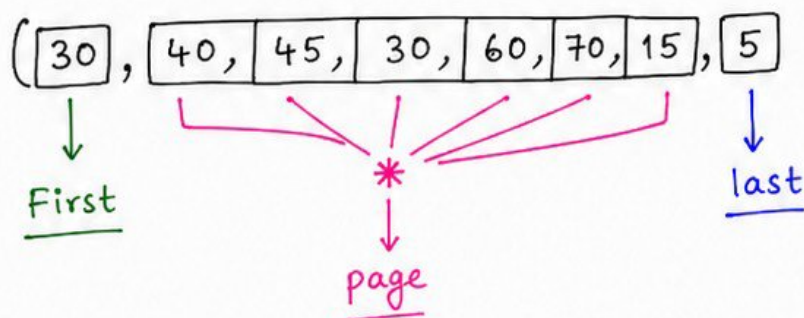
name, __, role, __ = person

print(name) o/p:- Maria

print(role) o/p:- DE

We can combine Asterisk (*) and (-).

Ex:-



Explore and Analyze :- Lists

Analyze :- Analyzing functions going to give you a quick snapshot to find high, low total and. (functions)

58

2	5	800
---	---	-----

 → max() →

800

 → Find extreme high.

800	2	300
-----	---	-----

 → min() →

2

 → Find extreme low.

2	3	5
---	---	---

 → sum() →

10

 → Find the total.

2	3	5
---	---	---

 → len() →

3

 → Find the length.

Completeness & existence check :-

✓ 'A'	✓ 'B'	✓ 'C'
----------	----------	----------

 → all() →

TRUE

 → Did everything pass?

✓ 'A'	✗ 'F'	✓ ''
----------	----------	---------

 → any() →

TRUE

 → Did something pass?

Search & count :- (methods)

'A'	'B'	'A'
-----	-----	-----

 → .count('A') →

2

 → How often?

'A'	'B'	'C'
-----	-----	-----

 → .index('A') →

0

 → where it appears?
0 1 2

membership & identity :- (operators)

'A'

 in

'A'	'B'
-----	-----

 →

TRUE

 → check if it exists?

'A'

 is

'A'	'B'
-----	-----

 →

FALSE

 → check if same object.

Comparison :- (operators)

'A'	'B'
-----	-----

 ==

'A'	'B'
-----	-----

 →

TRUE

.

'A'	'B'
-----	-----

 >

'A'	'B'
-----	-----

 →

FALSE

.

Changing Lists:-1. How to Add new items :-

We use `.append()` function.

→ Append the new value at end of the list.

```
letters = ['a', 'b', 'c', 'd']
```

TASK:- Add 'x' to the end of the list.

```
letters.append('x')      o/p:- ['a', 'b', 'c', 'd', 'x']
```

```
letters.append('y')      o/p:- ['a', 'b', 'c', 'd', 'x', 'y']
```

Insert At specific position.

Here we use `.insert()`

```
num = ['a', 'b', 'c', 'd']
```

```
print(num.insert(1, 'f'))      o/p:- ['a', 'f', 'b', 'c', 'd']
```

↑
(insert at index 1)

In 2D-Matrix :-

Add new row to the end of matrix.

```
MATRIX = [['a', 'b', 'c'], ['d', 'e', 'f'], ['g', 'h', 'i']]
```

→ Matrix Add new row at end of matrix.

```
matrix.append(['x', 'y', 'z'])
```

```
print(matrix)
```

→ Add New Row at the start of matrix.

```
matrix.insert(0, ['a', 'b', 'c', 'd'])
```

```
print(matrix)
```

IMP :-

Add 'x' at the end of second row.

```
matrix[1].append('x')
print(matrix)
```

Add 'z' at the start of first row.

```
matrix[0].insert(0, 'z')
```

Add 'y' at the 3rd row b/w any 2 elements.

```
matrix[2].insert(2, 'y')
```

SUMMARY :-

- `append(value)`: Add item at the end.
- `insert(index, value)`: Add item at specific position.

```
m = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
m.remove([1, 2, 3])
```

```
m.pop()
```

```
print(m)
```

```
m[1].remove('c') → specific element
```

REMOVE ITEMS :-1. REMOVE EVERYTHING :-

```
letters = ['a', 'b', 'c']
```

o/p:- []

```
print(letters.clear())
```

or

```
print(letters)
```

o/p:- []

2. SPECIFIC VALUE :- Delete

[by value → .remove()
by position → .pop()]

Remove:

```
letters = ['a', 'b', 'a']
```

```
print(letters.remove('a'))
```

o/p:- ['b', 'a']

```
" " " " " " )
```

o/p:- ['b']

- `pop()` :- Default → Remove last item, remove & returns item

```
le = ['a', 'b', 'c']
```

```
print(le.pop())
```

o/p:- 'c'

```
le = ['a', 'b', 'c']
```

```
removed = le.pop(0)
```

```
print(le)
```

o/p:- ['b', 'c']

```
print(removed)
```

o/p:- 'a'